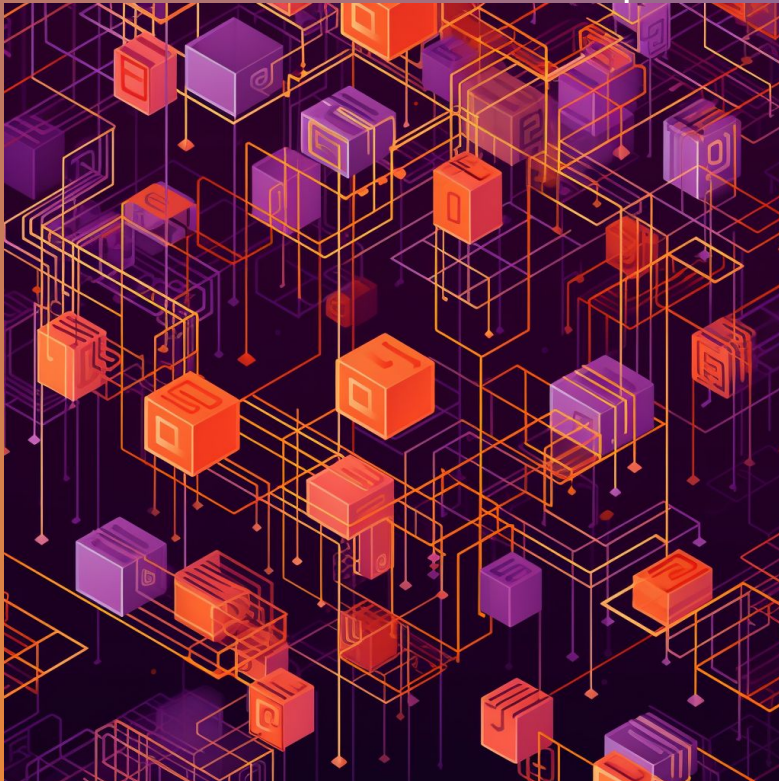


Blockchain et développement



Helder SALVADOR, Développeur
Blockchain et Directeur Pédagogique
filière Blockchain ESGI



SOMMAIRE

Pourquoi la blockchain ?

Fonctionnement de la blockchain

Cas d'utilisation

Smart-contracts

Solidity

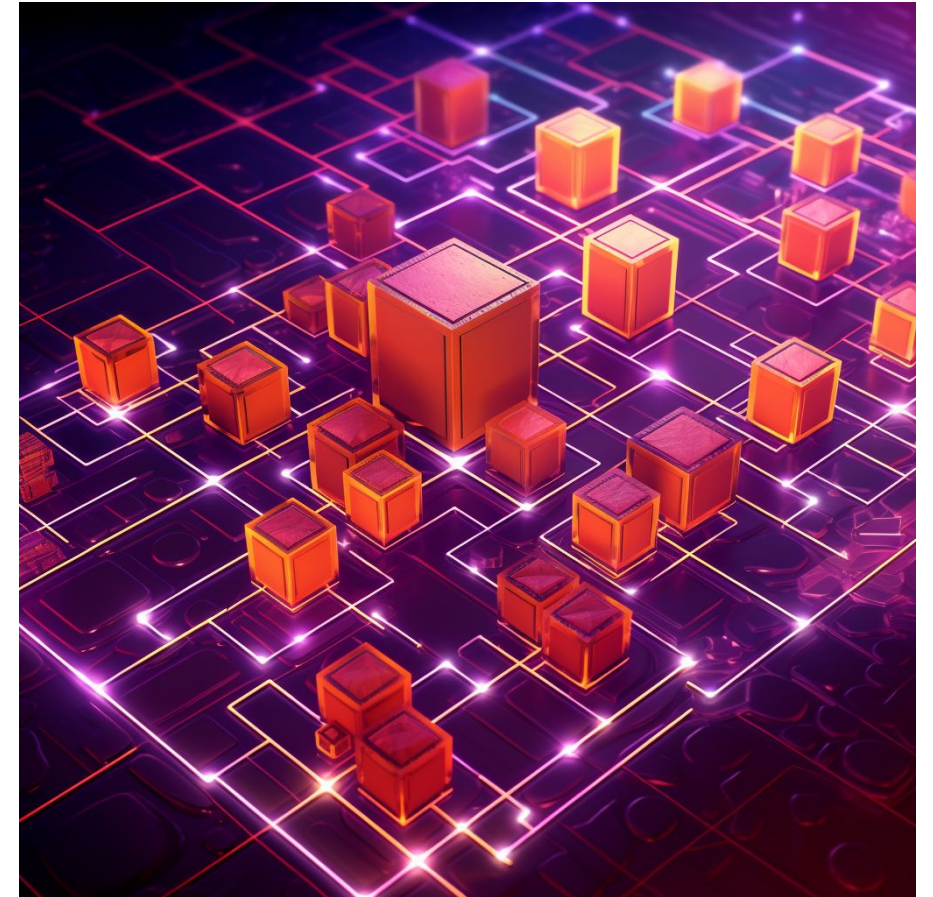
Design Patterns

Introduction

La blockchain Ethereum est une plateforme décentralisée basée sur la technologie de la blockchain.

Elle a été proposée par Vitalik Buterin en 2013 et lancée en 2015.

Contrairement à Bitcoin, qui a été conçu principalement pour être utilisé comme une monnaie numérique, Ethereum est une plateforme qui permet aux développeurs de créer et de déployer des applications décentralisées (dApps) en utilisant des contrats intelligents (smart contracts).

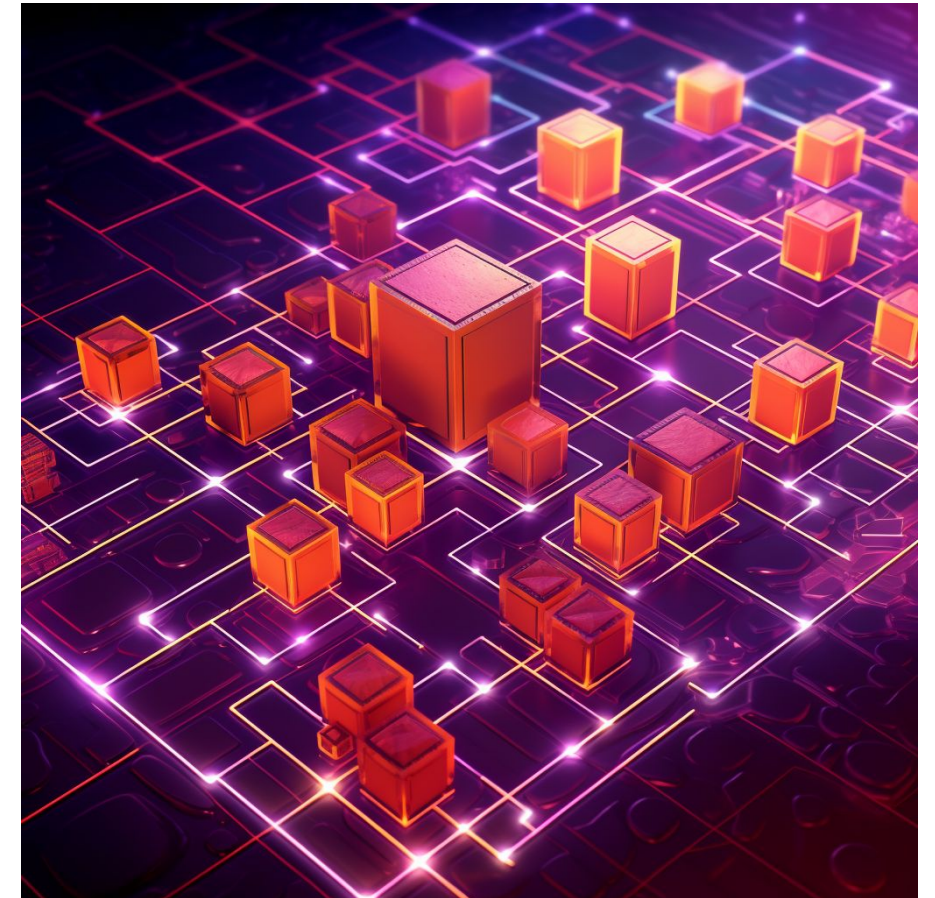


Introduction

Décentralisation : Ethereum est une blockchain décentralisée, ce qui signifie qu'aucune autorité centrale ne contrôle le réseau. Les données sont réparties sur un grand nombre d'ordinateurs (nœuds) dans le monde entier. Cela rend le réseau résistant à la censure et aux pannes.

Smart Contracts : Les contrats intelligents (smart contracts) sont des programmes autonomes qui s'exécutent automatiquement lorsque certaines conditions prédéfinies sont remplies. Ils sont écrits en langage de programmation spécifique appelé Solidity (similaire à JavaScript) et sont déployés sur la blockchain Ethereum. Les smart contracts permettent d'automatiser des actions et des transactions sans nécessiter d'intermédiaires.

Virtual Machine Ethereum (EVM) : L'Ethereum Virtual Machine (EVM) est un environnement d'exécution sécurisé pour les smart contracts sur la blockchain Ethereum. Elle permet aux nœuds de la blockchain de traiter les instructions du smart contract de manière cohérente sur l'ensemble du réseau.

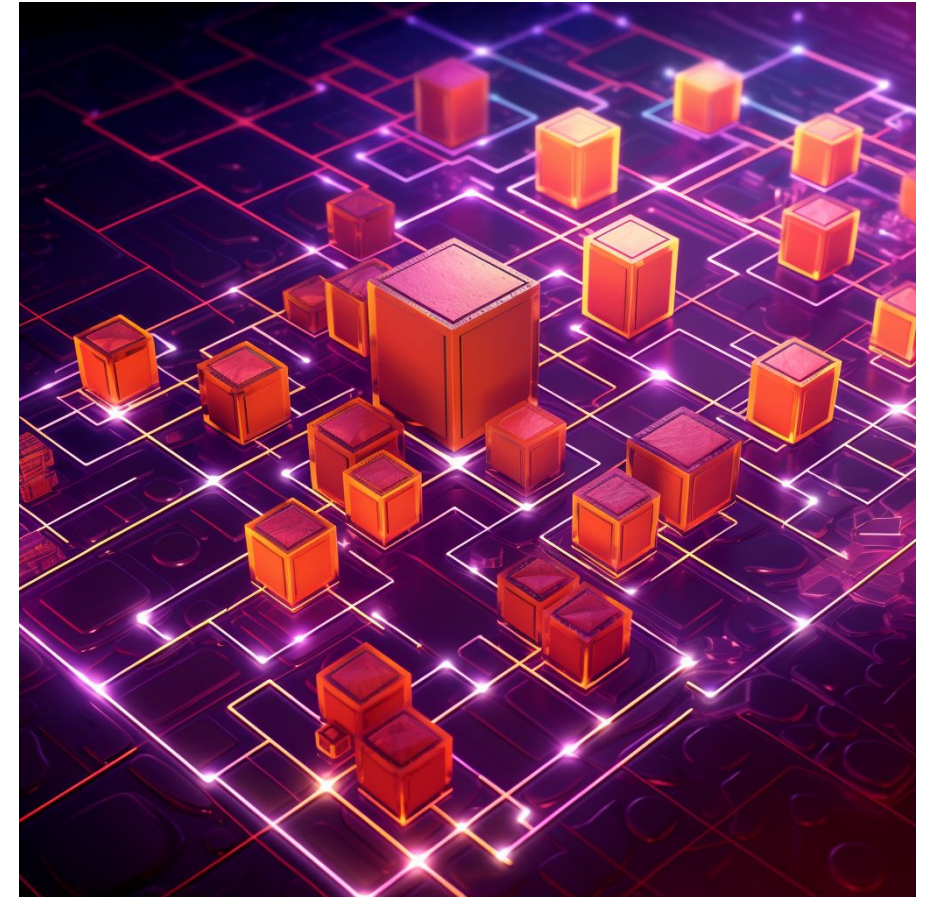


Introduction

Ether (ETH) : Ether est la cryptomonnaie native d'Ethereum. Elle est utilisée pour payer les frais de transaction sur le réseau et récompenser les mineurs qui sécurisent la blockchain en validant les transactions.

Frais de transaction : Chaque action effectuée sur Ethereum (comme l'exécution d'un smart contract en écriture ou l'envoi de fonds) nécessite le paiement de frais de transaction, connus sous le nom de "gas". Les frais de gas sont payés en ether et varient en fonction de la complexité de l'action.

Interopérabilité : Ethereum permet aux développeurs de créer des applications qui interagissent avec d'autres contrats intelligents. Cela signifie que différentes dApps peuvent communiquer et collaborer entre elles, créant ainsi des écosystèmes interconnectés.



Pourquoi la blockchain

?

Quels fondamentaux de l'informatique sont réglés?

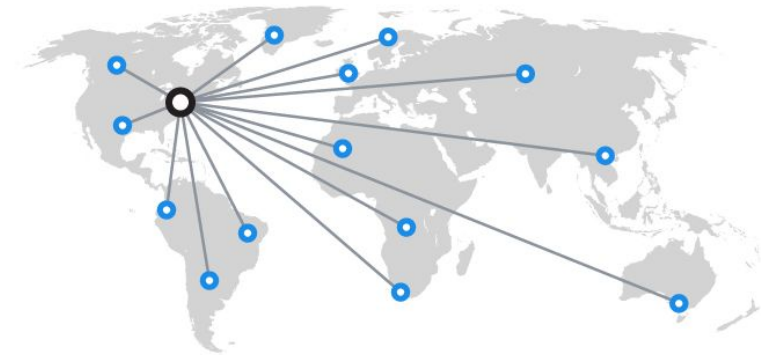
Elimination du tiers de confiance

Problème : Dans les systèmes financiers traditionnels, les transactions nécessitent des intermédiaires de confiance tels que les banques, les institutions de paiement ou les gouvernements pour valider et vérifier les échanges entre deux parties.

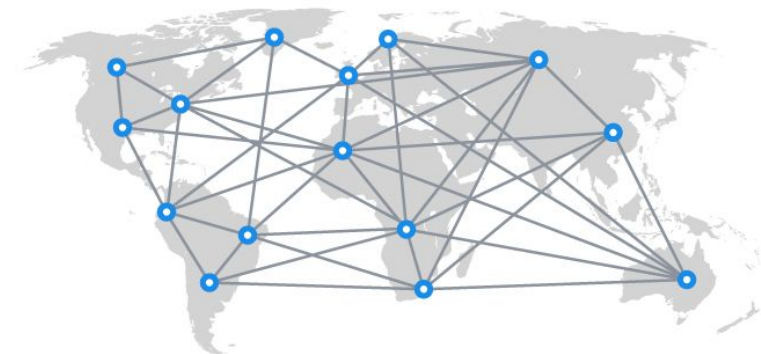
Solution apportée par Bitcoin : Bitcoin permet de réaliser des transactions de pair à pair (P2P) sans passer par un intermédiaire de confiance. La blockchain de Bitcoin, qui fonctionne sur un réseau distribué, remplace ces intermédiaires en offrant un mécanisme de consensus décentralisé où chaque participant peut vérifier la validité des transactions.

Impact : Cela réduit les frais de transaction, élimine le besoin de vérifier l'identité de l'autre partie via un tiers et permet des paiements transfrontaliers rapides.

Centralized Network



Decentralized Network



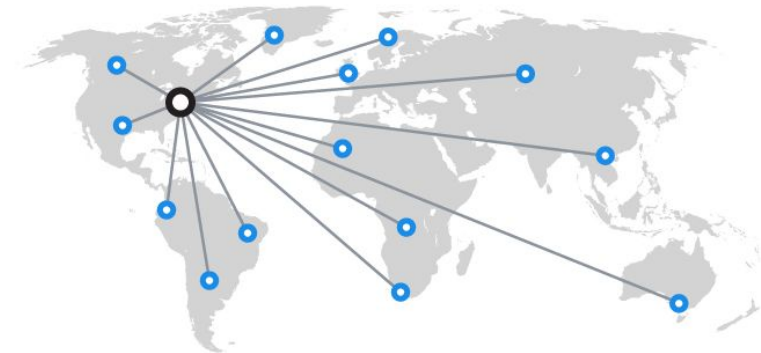
Elimination du tiers de confiance (2)

Problème : Les systèmes de paiement traditionnels, en particulier pour les transactions internationales, peuvent entraîner des frais élevés et des délais de traitement longs (jusqu'à plusieurs jours).

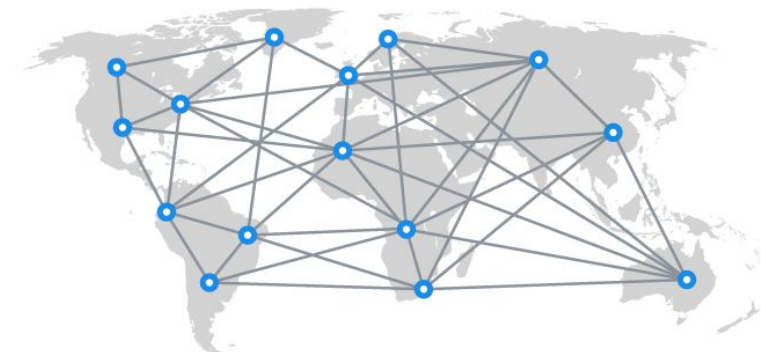
Solution apportée par Bitcoin : En permettant des transactions directes entre les utilisateurs sans passer par des intermédiaires, Bitcoin peut réduire les coûts de transaction et le temps nécessaire pour traiter les paiements. Les transactions sur le réseau Bitcoin sont vérifiées par les mineurs et peuvent être confirmées en 10 minutes environ (le temps moyen de création d'un bloc).

Impact : Cela rend les transactions plus accessibles, surtout dans les régions où les services bancaires sont coûteux ou limités.

Centralized Network



Decentralized Network



Double-spending



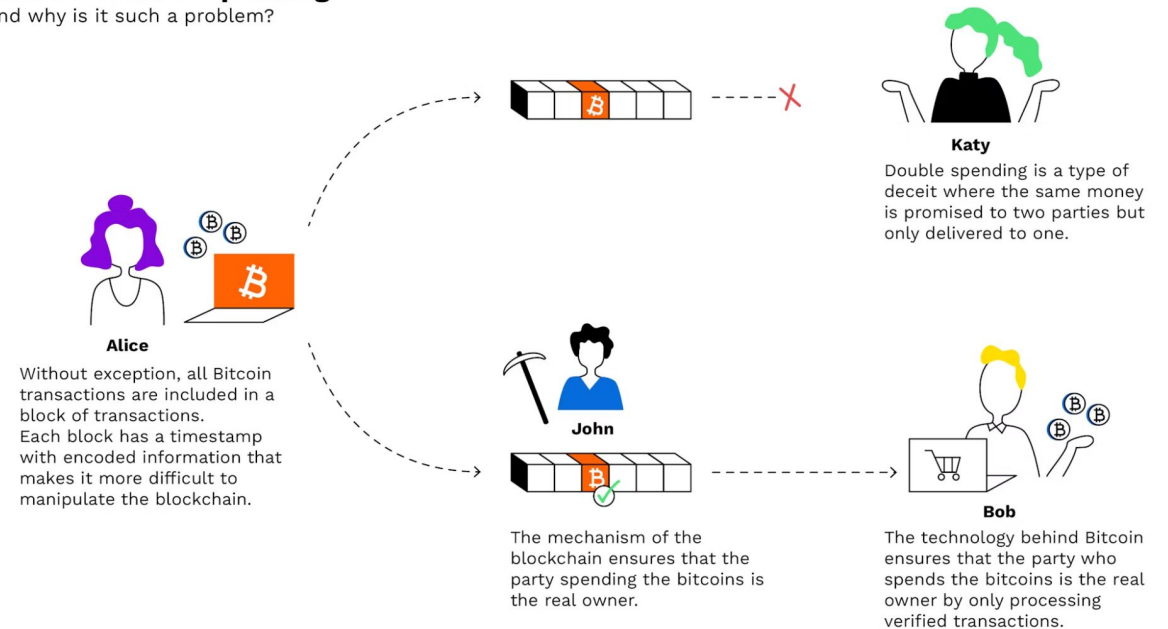
Problème : Dans les systèmes de monnaie numérique, le risque de double dépense est important. Cela signifie qu'un même montant pourrait être dépensé plus d'une fois si le système ne dispose pas d'un moyen efficace pour vérifier qu'une unité de monnaie n'a pas été déjà utilisée.

Solution apportée par Bitcoin : Bitcoin utilise une blockchain où chaque transaction est enregistrée de manière immuable et vérifiée par les mineurs. Le mécanisme de consensus basé sur le Proof of Work (PoW) garantit que chaque transaction est validée et ajoutée à la chaîne de manière à éviter que le même bitcoin soit dépensé plus d'une fois.

Impact : Cela assure l'intégrité du système et empêche les utilisateurs malveillants de manipuler les enregistrements pour dupliquer les fonds.

What is Double Spending

and why is it such a problem?



Inclusion Financière et Accessibilité



Problème : Dans de nombreuses régions du monde, les personnes n'ont pas accès aux services bancaires traditionnels. Cela limite leur capacité à envoyer, recevoir ou stocker de la valeur de manière sécurisée.

Solution apportée par Bitcoin : Bitcoin offre un moyen pour toute personne ayant accès à Internet de stocker, envoyer et recevoir de la valeur de manière sécurisée sans avoir besoin d'un compte bancaire. Il permet de créer un portefeuille de cryptomonnaies sans procédure compliquée ni restrictions géographiques.

Impact : Bitcoin peut faciliter l'inclusion financière et offrir des options aux populations non bancarisées, en leur permettant de participer à l'économie numérique mondiale.



Transparence et Traçabilité



Problème : Les systèmes financiers traditionnels manquent souvent de transparence, rendant difficile la traçabilité des fonds, ce qui peut encourager la fraude ou la corruption.

Solution apportée par Bitcoin : La blockchain de Bitcoin est publique, ce qui signifie que toutes les transactions peuvent être consultées par n'importe qui. Bien que les identités des utilisateurs soient pseudonymes, la transparence de la blockchain permet de vérifier facilement l'historique des transactions.

Impact : Cela améliore la transparence financière et permet de suivre les mouvements de fonds, par exemple, dans des organisations caritatives ou des fonds publics.





Fonctionnement de la blockchain



Protocole / Transaction / Réseau

Protocole de consensus



Un **protocole de consensus** est un ensemble de règles et de processus utilisés par les participants d'un réseau décentralisé (comme une blockchain) pour parvenir à un accord sur l'état des données ou des transactions qui doivent être enregistrées dans le registre partagé.

En d'autres termes, il s'agit du mécanisme qui permet aux différents nœuds (ordinateurs) d'un réseau distribué de s'assurer qu'ils ont tous la même version de la blockchain, sans qu'un tiers de confiance soit nécessaire.

Preuve de travail

vs

Preuve d'enjeu



Le premier mineur à résoudre le puzzle asymétrique est sélectionné. Les mineurs se font concurrence pour résoudre le puzzle.



Équipements spécialisés nécessaires pour optimiser la puissance de traitement.



Investissement initial pour acheter le matériel.



Consommation d'énergie élevée.



Un processus de sélection déterministe est utilisé. Les mineurs se font concurrence pour être sélectionnés.



Une unité de serveur classique est souvent (largement) suffisante.



Investissement initial pour acheter les fonds mis en staking et bâtir une réputation.



Consommation d'énergie normale.

Protocole de consensus



+



Proof of Work (PoW) :

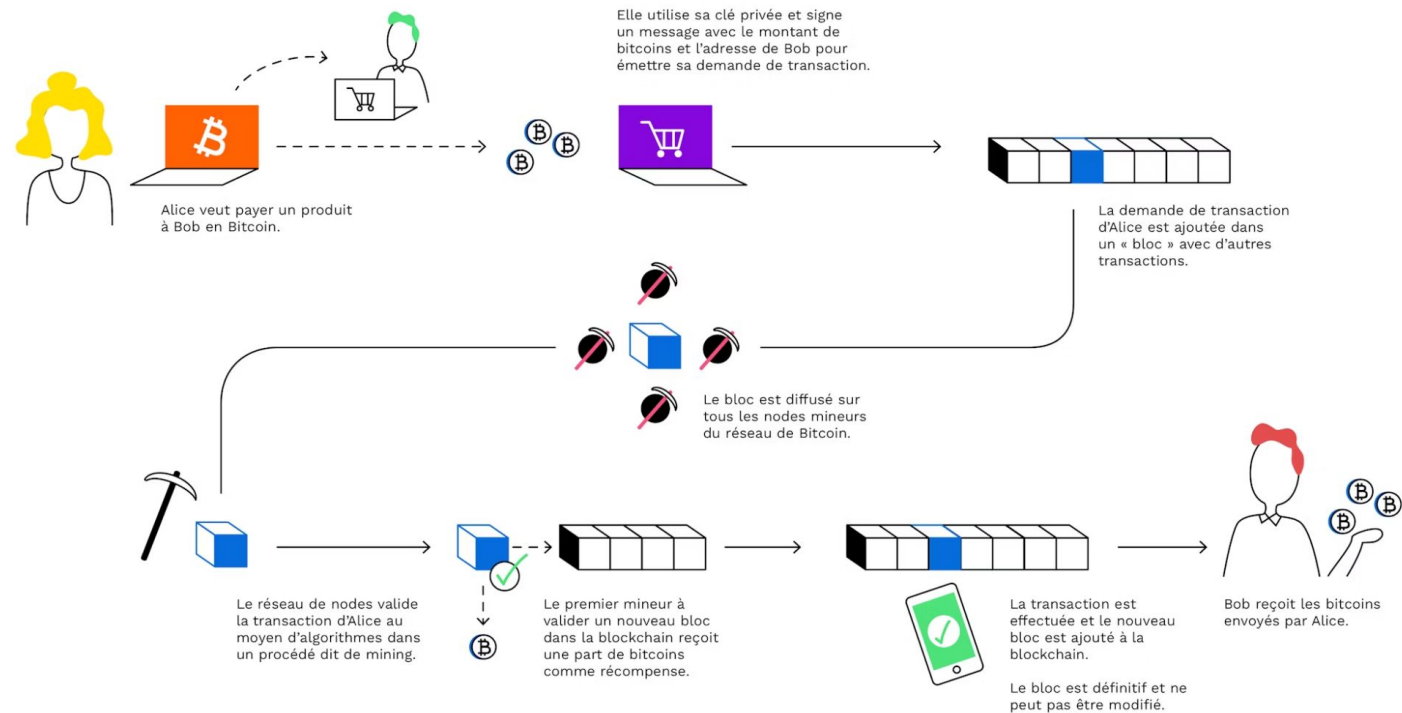
- **Description** : Les nœuds (appelés mineurs) résolvent des problèmes mathématiques complexes pour valider les transactions et créer de nouveaux blocs. Le premier mineur à résoudre le problème propose le nouveau bloc, et les autres nœuds le vérifient. Bitcoin et Ethereum (avant sa transition vers PoS) utilisent ce protocole.
- **Avantages** : Sécurité élevée, résistant aux attaques.
- **Inconvénients** : Consommation énergétique très élevée et vitesse de transaction limitée.

Proof of Stake (PoS) :

- **Description** : Dans le PoS, les validateurs (qui remplacent les mineurs) sont choisis pour créer de nouveaux blocs en fonction de la quantité de cryptomonnaie qu'ils possèdent et qu'ils mettent en jeu (stake). Plus un validateur a de cryptomonnaies en jeu, plus il a de chances de proposer un bloc.
- **Avantages** : Consommation d'énergie réduite, vitesse de transaction plus élevée.
- **Inconvénients** : Risque de concentration du pouvoir dans les mains des détenteurs de grandes quantités de cryptomonnaies.

La blockchain et le minage

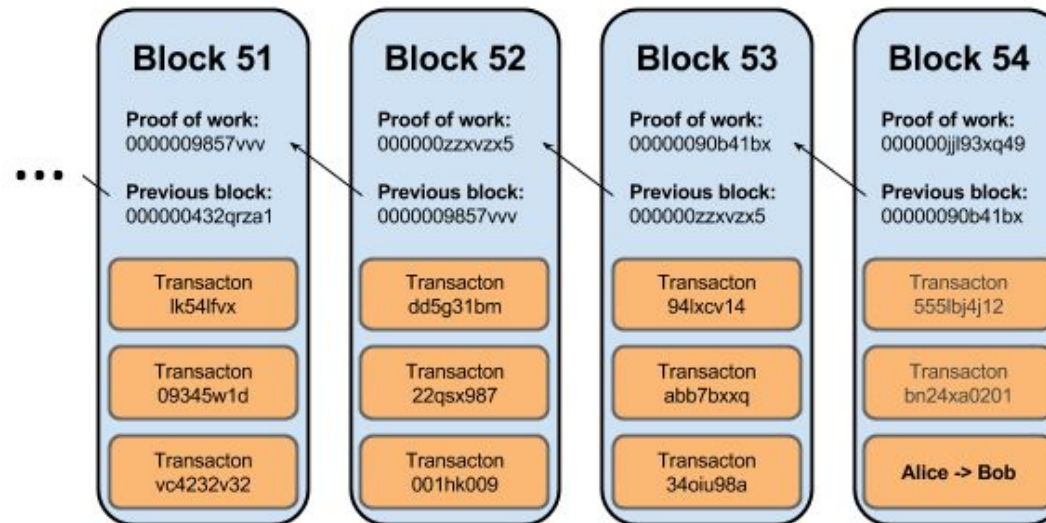
Qu'est-ce que le minage de bitcoin ? (comment fonctionne une transaction de bitcoins)



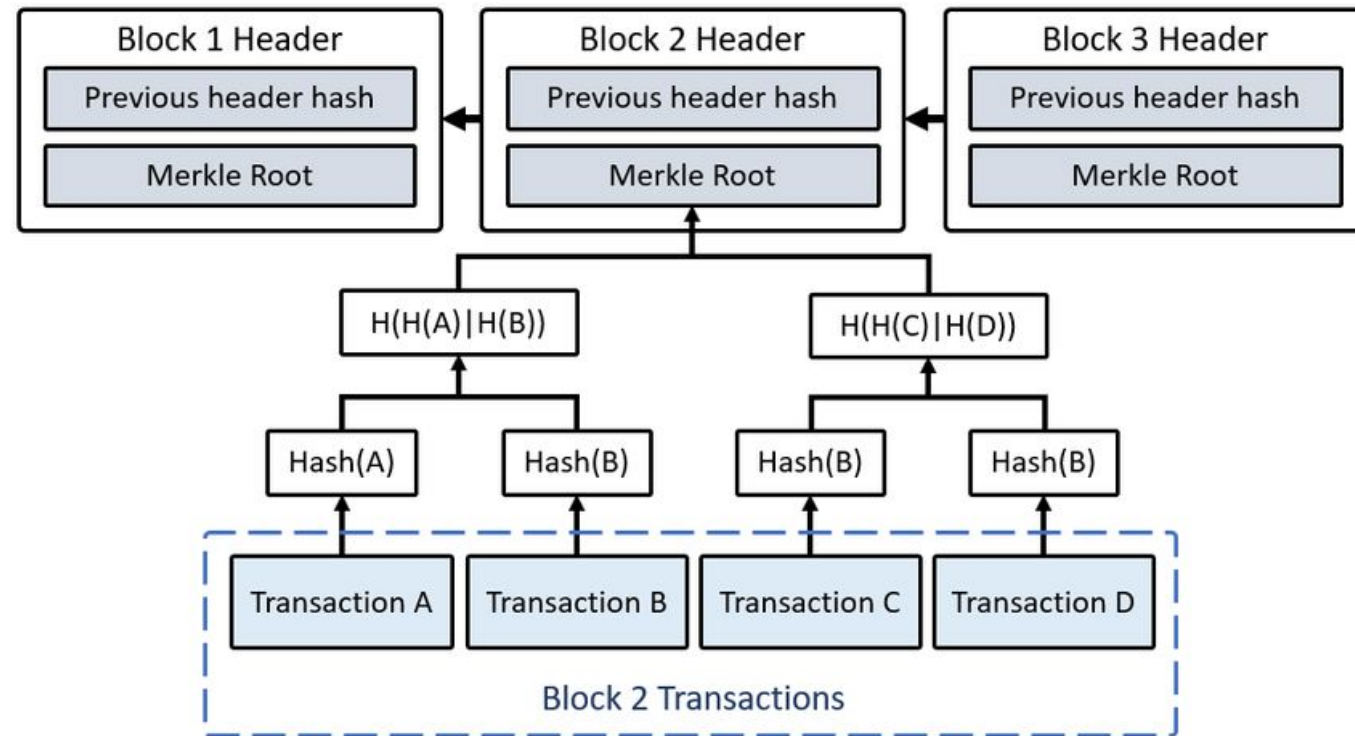
La "chaîne de bloc"



Blockchain - "Livre des comptes"



L'arbre de Merkle

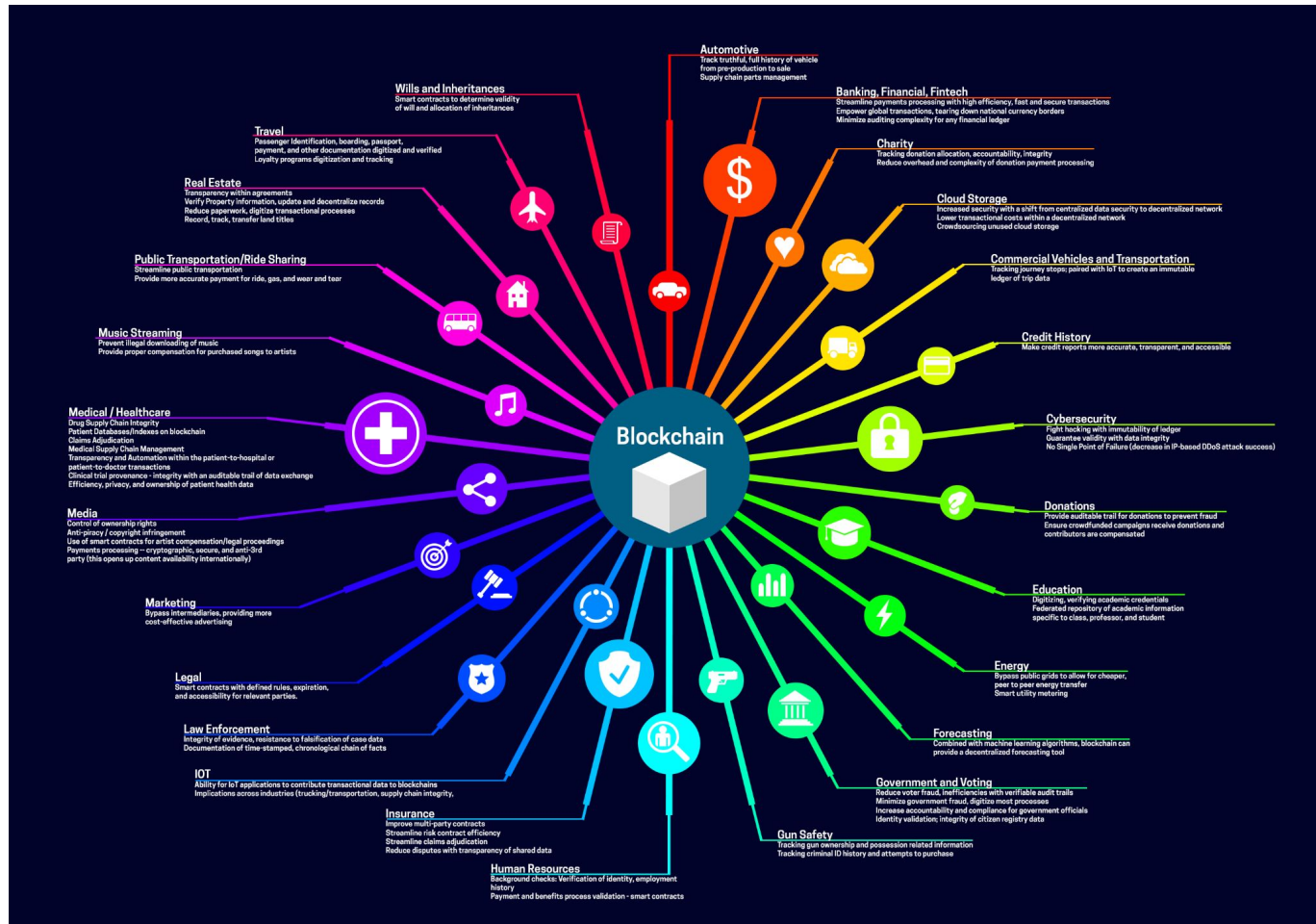




Cas d'utilisation

Quels domaines sont touchés ?

Qui est touché ?



Pourquoi c'est partout ?



Transparence et Traçabilité

- Chaque transaction sur une blockchain est enregistrée de manière immuable dans un registre distribué. Cela garantit une transparence totale et permet de suivre l'historique complet des transactions.

Décentralisation

- La blockchain permet d'éliminer les intermédiaires dans les transactions (comme les banques ou les notaires), ce qui réduit les coûts et les délais.
- Chaque participant (nœud) a une copie du registre, ce qui renforce la résilience et la disponibilité des données.

Sécurité Renforcée

- Grâce à la cryptographie et à la nature décentralisée de la blockchain, il est difficile de falsifier les transactions ou de manipuler les données.
- Une fois qu'une transaction est validée et enregistrée dans un bloc, il est pratiquement impossible de la modifier sans le consensus du réseau.

Automatisation avec les Smart Contracts

- Les smart contracts permettent d'automatiser des transactions dès que certaines conditions sont remplies, ce qui améliore l'efficacité des processus.
- Cela élimine le besoin de vérifications manuelles et réduit le risque d'erreur humaine.

Pourquoi ce n'est pas partout ?



Consommation Énergétique

- Les blockchains basées sur le Proof of Work (PoW) comme Bitcoin consomment une grande quantité d'énergie en raison des calculs complexes nécessaires pour valider les transactions.

Complexité Technique

- La compréhension et l'implémentation de la blockchain nécessitent des compétences techniques avancées, rendant son adoption difficile pour des petites entreprises ou des projets non techniques.

Scalabilité Limitée

- Les blockchains souffrent de problèmes de scalabilité, ce qui signifie que la vitesse de traitement des transactions est limitée par la taille des blocs et le temps de validation.

Problèmes de Sécurité sur les Smart Contracts

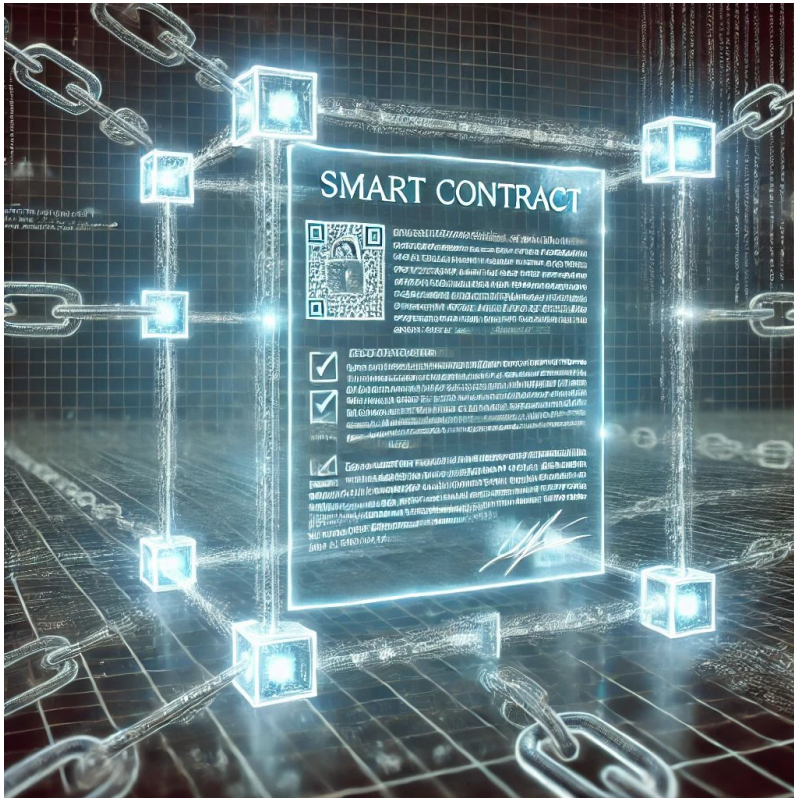
- Bien que la blockchain elle-même soit sécurisée, les smart contracts peuvent contenir des vulnérabilités de code.
- Les transactions sont irréversibles. Si une erreur est commise (par exemple, une adresse incorrecte), il est impossible de récupérer les fonds.



Smart-contracts

A quoi cela ressemble ?

D'après ChatGPT :



D'après la vraie vie :

```
1 pragma solidity 0.6.11;
2
3 contract VendingMachine {
4
5     // Declare state variables of the contract
6     address public owner;
7     mapping (address => uint) public cupcakeBalances;
8
9     // When 'VendingMachine' contract is deployed:
10    // 1. set the deploying address as the owner of the contract
11    // 2. set the deployed smart contract's cupcake balance to 100
12    constructor() public {
13        owner = msg.sender;
14        cupcakeBalances[address(this)] = 100;
15    }
16
17    // Allow the owner to increase the smart contract's cupcake balance
18    function refill(uint amount) public {
19        require(msg.sender == owner, "Only the owner can refill.");
20        cupcakeBalances[address(this)] += amount;
21    }
22
23    // Allow anyone to purchase cupcakes
24    function purchase(uint amount) public payable {
25        require(msg.value >= amount * 1 ether, "You must pay at least 1
26        ETH per cupcake");
27        require(cupcakeBalances[address(this)] >= amount, "Not enough
28        cupcakes in stock to complete this purchase");
29        cupcakeBalances[address(this)] -= amount;
30        cupcakeBalances[msg.sender] += amount;
31    }
32 }
```

C'est quoi ?

Automatisation : Les smart contracts automatisent les processus en exécutant des actions prédéfinies, comme le transfert de fonds, une fois que les conditions sont remplies.

Transparence : Le code des smart contracts est visible par tous sur la blockchain, ce qui permet à toutes les parties impliquées de comprendre les règles de l'accord.

Inaltérabilité : Une fois déployé sur la blockchain, le code du smart contract et son état sont immuables, garantissant que les termes ne peuvent pas être modifiés sans l'accord des parties.

Automatisation des Transactions : Ils permettent d'automatiser des transactions financières (paiements, transferts de fonds) sans nécessiter de tiers de confiance.

Sécurisation des Accords : Ils garantissent que les conditions d'un accord sont respectées avant qu'une action ne soit déclenchée, par exemple dans des applications de crowdfunding, d'assurance ou de prêt.

Création de dApps (Applications Décentralisées) : Les smart contracts sont à la base des applications décentralisées (dApps) qui fonctionnent sur des blockchains comme Ethereum, offrant des services financiers, de jeux, de gestion de données, etc.

Gestion des Actifs Numériques : Ils facilitent la création et le transfert de tokens (cryptomonnaies, NFTs), la gestion de droits numériques et la traçabilité des actifs sur la blockchain.



Solidity

Le premier langage de programmation de smart-contract



TYPES





Uint

uint8/ 16/ 32/ 64/ 128/ 256 : Entier non signé de 256 bits (le plus utilisé)

Valeur maximum de l'uint256:

115792089237316195423570985008
687907853269984665640564039457
584007913129639935

Nous n'utilisons pas d'int car il n'est jamais nécessaire d'utiliser des nombres négatifs. (ex : On ne peut pas posséder -10 Ether)



Adresses

address : Adresse Ethereum de 20 octets
(peut être un wallet ou un contrat)

Ce type est utilisé pour enregistrer par exemple, qui est un administrateur du smart-contract ou qui a déjà utilisé celui-ci

address payable : Adresse Ethereum de 20 octets qui peut recevoir de l'ether

Le mot clé payable est un peu vicieux car il est nécessaire pour recevoir de l'Ether (la cryptomonnaie du réseau) mais pas nécessaire pour le reste. Il est assez rare de le croiser de nos jours



String et bytes

string : Chaîne de caractères UTF-8.

Attention le string est type assez lourd et utilisé pour des chaînes de caractères très courtes. (Problème de coût)

bytesX : Tableau de bytes de taille fixe où X est compris entre 1 et 32.

Ce type est très utile notamment pour des signatures d'adresse ou pour un système de droit



Tableaux et mapping

fixed-size arrays : Tableau de taille fixe (e.g., `uint[5]`).

Attention les tableaux peuvent être dynamiques mais seulement dans les fonctions

Ex: `uint balance[] = new uint[](size);`

mapping : Structure de données associant des clés à des valeurs (e.g., `mapping(address => uint)`).

Point intéressant, les mappings peuvent s'imbriquer mais veuillez à ne pas en imbriquer plus de deux, la logique utilisée serait à revoir si jamais,

VISIBILITÉ ET FONCTIONS



Visibilité

Le mot-clé `external` est l'une des quatre options de visibilité disponibles en Solidity, avec `public`, `private` et `internal`.

Chaque option a un effet sur la manière dont la fonction peut être appelée à l'intérieur du contrat lui-même et depuis l'extérieur (d'autres contrats ou adresses).

```
contract MyContract {  
    uint256 public myNumber;  
  
    // Fonction externe qui peut être appelée depuis l'extérieur du contrat  
    function setNumber(uint256 _num) external {  
        myNumber = _num;  
    }  
  
    // Fonction publique qui ne peut pas appeler la fonction externe  
    function incrementNumber() public {  
        // Cette ligne générera une erreur de compilation car la fonction externe ne peut pas être appelée ici  
        // setNumber(myNumber + 1);  
    }  
}
```

Public : accessible à tous, de n'importe où

Private : accessible par le contrat où elle se trouve (variable ou fonction d'ailleurs)

Internal : accessible par le contrat où elle se trouve + les contrats qui hérite du contrat original

External : Seulement par des contrats extérieurs



Fonctions

Payable : De l'éther va être envoyé lors de l'appel de la fonction (msg.value est une variable qui permet de savoir combien est envoyé)

View : Permet de préciser au compilateur que la fonction ne va faire que retourner des valeurs, son utilisation est gratuite dans ce cas.

Returns : Non obligatoire mais permettant de préciser le type de la fonction et le nombre de variable et de type retourné.
Nous pouvons donc return un string, un bool et un tableau d'adresse si on le souhaite.



Fonctions

Le mot-clé “pure” est utilisé en Solidity pour déclarer une fonction qui n'accède à aucun état du contrat et ne modifie pas l'état non plus.

Une fonction marquée comme pure est utilisée pour effectuer des calculs ou des traitements sur les paramètres qu'elle reçoit et renvoyer un résultat, mais elle ne peut pas modifier les variables d'état du contrat ni lire d'informations à partir de celles-ci.

- **Aucun accès à l'état :** Une fonction pure ne peut pas lire ou modifier les variables déclarées dans le contrat (variables d'état) ni appeler d'autres fonctions qui modifient l'état. Elle est limitée à des calculs locaux basés uniquement sur les paramètres qu'elle reçoit.
- **Calculs basés sur les paramètres :** Une fonction pure est généralement utilisée pour effectuer des calculs sur les paramètres qu'elle reçoit. Elle peut retourner des valeurs basées uniquement sur ces paramètres.
- **Pas de frais de gas pour l'accès à l'état :** Étant donné qu'une fonction pure n'accède pas à l'état du contrat, son exécution ne nécessite pas de frais de gas supplémentaires liés à l'accès à l'état.
- **Utilise pour des calculs mathématiques majoritairement**



Paramètres de fonctions

Storage est un mot-clé utilisé pour déclarer des variables de type référence (comme les mappings et les variables de type struct) qui doivent être stockées de manière permanente sur la blockchain.

- Les données stockées dans storage sont persistantes entre les différentes exécutions de la fonction et même après la fin de l'appel du contrat.
- Les modifications apportées aux variables storage sont immédiatement visibles sur la blockchain et ont un impact permanent sur l'état du contrat.
- Les variables storage occupent de l'espace sur la blockchain et entraînent des frais de gas plus élevés lors de la modification de leur valeur.



Paramètres de fonctions

Memory est un mot-clé utilisé pour déclarer des variables locales temporaires à l'intérieur d'une fonction.

- Les données stockées dans memory ne persistent pas au-delà de l'exécution de la fonction et sont nettoyées après son exécution.
- Les variables memory sont principalement utilisées pour stocker des données temporaires lors de l'exécution de fonctions et de leur traitement.
- Les paramètres des fonctions et les variables locales sont généralement stockés dans memory.
- Il est obligatoire pour les types lourds, tels que string, struct ou des tableaux.



Paramètres de fonctions

Calldata est un mot-clé utilisé pour déclarer des paramètres de fonction qui représentent des données passées à une fonction depuis l'extérieur du contrat.

- Les données dans calldata sont en lecture seule, ce qui signifie que la fonction ne peut pas modifier les valeurs des paramètres calldata.
- Les paramètres calldata sont utilisés lorsque vous souhaitez passer des données à une fonction sans que cela n'affecte l'état du contrat et sans gas supplémentaire pour les copier dans memory ou storage.
- Les fonctions avec des paramètres calldata sont typiquement utilisées dans les contrats lorsque des données doivent être fournies depuis l'extérieur.



FONCTIONS EXEMPLE

DESIGN PATTERNS

```
// To buy a listed item, with WETH only
function buyItem(uint _id) payable{
    require(items[_id].listed, "Do not try to buy an unlisted item");
    require(balanceOf(address(this), _id) > 0, "There is an issue, the NFT is not owned by the contract");
    require(msg.value >= items[_id].price, "Not enough balance");

    transfer(items[_id].seller, items[_id].price);
    safeTransferFrom(address(this), _msgSender(), _id, 1, "");
}
```

```
// Returns all listed item
function getAllListedItem() public view returns(Item[] memory){
    if(_itemCounter.current() != 0){
        Item[] memory listedItems = new Item[](_itemCounter.current() - 1);
        for(uint i = 0; i < _itemCounter.current(); i++){
            if(items[i].listed == true){
                listedItems[i] = items[i];
            }
        }
        return listedItems;
    }

    Item[] memory noItems = new Item[](0);
    return noItems;
}
```

**ERC (=ETHEREUM
REQUEST FOR COMMENT)**

Normes Ethereum

ERC-20 : C'est le standard le plus populaire pour les jetons sur la blockchain Ethereum. Il définit les interfaces de base pour créer et gérer des jetons fongibles, ce qui signifie que chaque unité de jeton est identique et interchangeable. La plupart des jetons que vous trouvez sur les échanges sont des jetons ERC-20.

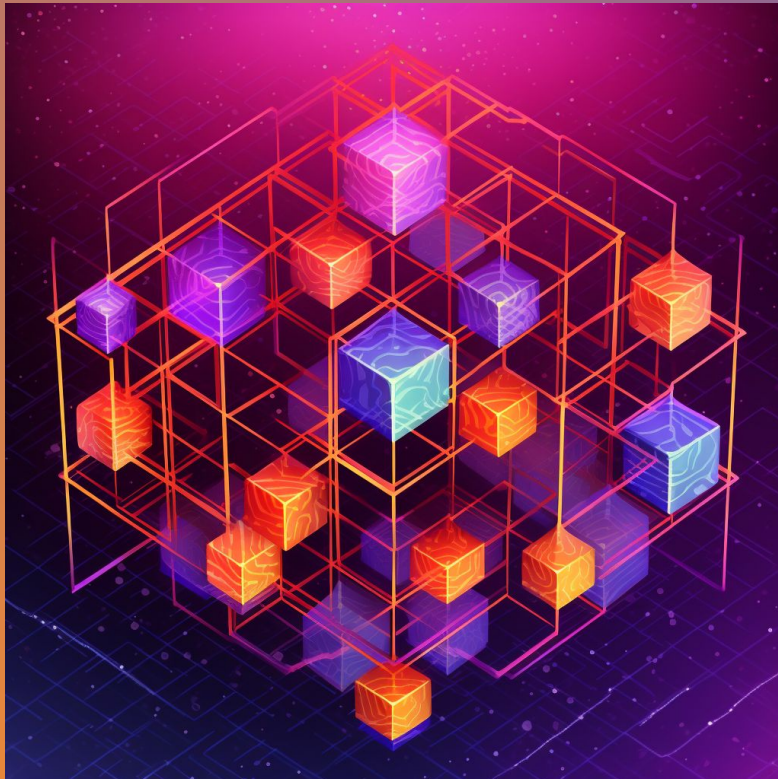
ERC-721 : Également connu sous le nom de jetons non fongibles (NFTs), l'ERC-721 définit un standard pour la création de jetons uniques et indivisibles. Chaque unité de NFT a une valeur distincte et ne peut pas être remplacée par une autre. Cela les rend idéaux pour la représentation de biens numériques uniques tels que des œuvres d'art, des propriétés virtuelles ou des objets de collection.

ERC-1155 : Ce standard combine les caractéristiques des jetons ERC-20 et ERC-721. Il permet de créer des jetons fongibles et non fongibles dans un seul contrat. Cela facilite la création de collections de NFTs et de jetons fongibles au sein d'un même écosystème.





DESIGN PATTERNS



PATTERN DU REGISTRE

Le pattern du registre (Registry) permet de conserver une liste des contrats intelligents déployés sur la blockchain et fournit des fonctionnalités pour les enregistrer, les supprimer ou les mettre à jour.

Ce contrat de registre agit comme une base de données centralisée pour enregistrer les informations sur les contrats intelligents.



Pattern du registre

Enregistrement des contrats intelligents
: Lorsqu'un nouveau contrat intelligent est déployé, son adresse et d'autres informations pertinentes sont enregistrées dans le contrat de registre.

Accès aux informations des contrats :
Les utilisateurs ou d'autres contrats peuvent accéder au contrat de registre pour obtenir des informations sur les contrats intelligents enregistrés.
(adresses, les propriétaires, les paramètres ou toute autre donnée pertinente)





PATTERN DU CONTRÔLE D'ACCÈS

Le pattern du contrôle d'accès (Access Control) est utilisé pour gérer les niveaux d'accès et les autorisations des utilisateurs dans les contrats intelligents. Il permet de définir et de contrôler les privilèges des parties prenantes, en garantissant que seules les entités autorisées peuvent effectuer certaines actions spécifiques.

Définition des rôles et des autorisations : Par exemple, on peut avoir des rôles tels que "administrateur", "membre" ou "utilisateur" avec des autorisations spécifiques comme "créer", "modifier" ou "supprimer".



Pattern du contrôle d'accès (Access Control)

Attribution des rôles aux entités :

Les rôles sont attribués aux différentes entités impliquées dans le contrat intelligent. Cela peut inclure les adresses des comptes ou les contrats eux-mêmes.

Vérification des autorisations :

Avant d'exécuter une action spécifique, le contrat intelligent vérifie les autorisations de l'entité qui effectue l'appel. Si l'entité a les autorisations requises pour effectuer l'action, celle-ci est exécutée avec succès. Sinon, une exception est levée ou l'action est refusée.



PATTERN DU TRANSFERT DE VALEUR(TOKEN)



Il est largement utilisé dans les contrats intelligents pour créer et gérer des jetons numériques sur la blockchain. Ce pattern permet de représenter et de transférer des actifs numériques, tels que des cryptomonnaies ou des actifs virtuels, de manière sécurisée et traçable.

Création du contrat de jeton : Ce contrat de jeton définit les règles et les fonctionnalités liées aux jetons numériques, y compris leur création, leur transfert et leur gestion.



Pattern du transfert de valeur (token)

Émission des jetons :

Le contrat de jeton fournit une fonction spéciale pour l'émission initiale des jetons. Cette fonction est généralement appelée "mint" et permet de créer une quantité spécifique de jetons et de les attribuer à une adresse ou à un compte.

Transfert des jetons :

Les détenteurs de jetons peuvent transférer leurs jetons à d'autres adresses ou comptes en appelant la fonction de transfert du contrat de jeton.

Vérification des soldes :

Le contrat de jeton permet aux utilisateurs de vérifier les soldes de jetons d'une adresse ou d'un compte spécifique. Cela permet de suivre et de vérifier la propriété des jetons.





Pattern du transfert de valeur (token)

Émission des jetons :

Le contrat de jeton fournit une fonction spéciale pour l'émission initiale des jetons. Cette fonction est généralement appelée "mint" et permet de créer une quantité spécifique de jetons et de les attribuer à une adresse ou à un compte.

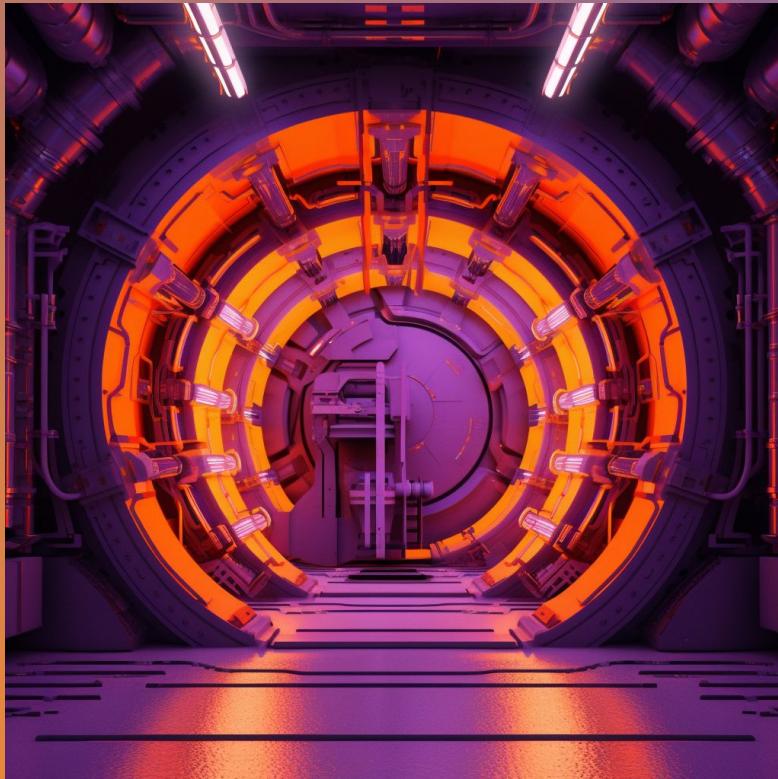
Transfert des jetons :

Les détenteurs de jetons peuvent transférer leurs jetons à d'autres adresses ou comptes en appelant la fonction de transfert du contrat de jeton.

Vérification des soldes :

Le contrat de jeton permet aux utilisateurs de vérifier les soldes de jetons d'une adresse ou d'un compte spécifique. Cela permet de suivre et de vérifier la propriété des jetons.





PATTERN DU COFFRE-FORT

Le pattern du coffre-fort (Vault) est utilisé pour sécuriser les fonds ou les actifs numériques dans un contrat intelligent. Il met en œuvre des mécanismes de sécurité pour protéger les actifs contre les accès non autorisés et les tentatives de vol.

Création du contrat de coffre-fort : Ce contrat est responsable de la gestion et de la sécurité des fonds ou des actifs numériques.

Pattern du coffre-fort (Vault)

Dépôt des fonds ou des actifs :

Les utilisateurs peuvent déposer leurs fonds ou leurs actifs numériques dans le contrat de coffre-fort en appelant une fonction spécifique, généralement appelée "deposit". Cette fonction enregistre les fonds et les associe à l'adresse de l'utilisateur.

Vérification de la propriété :

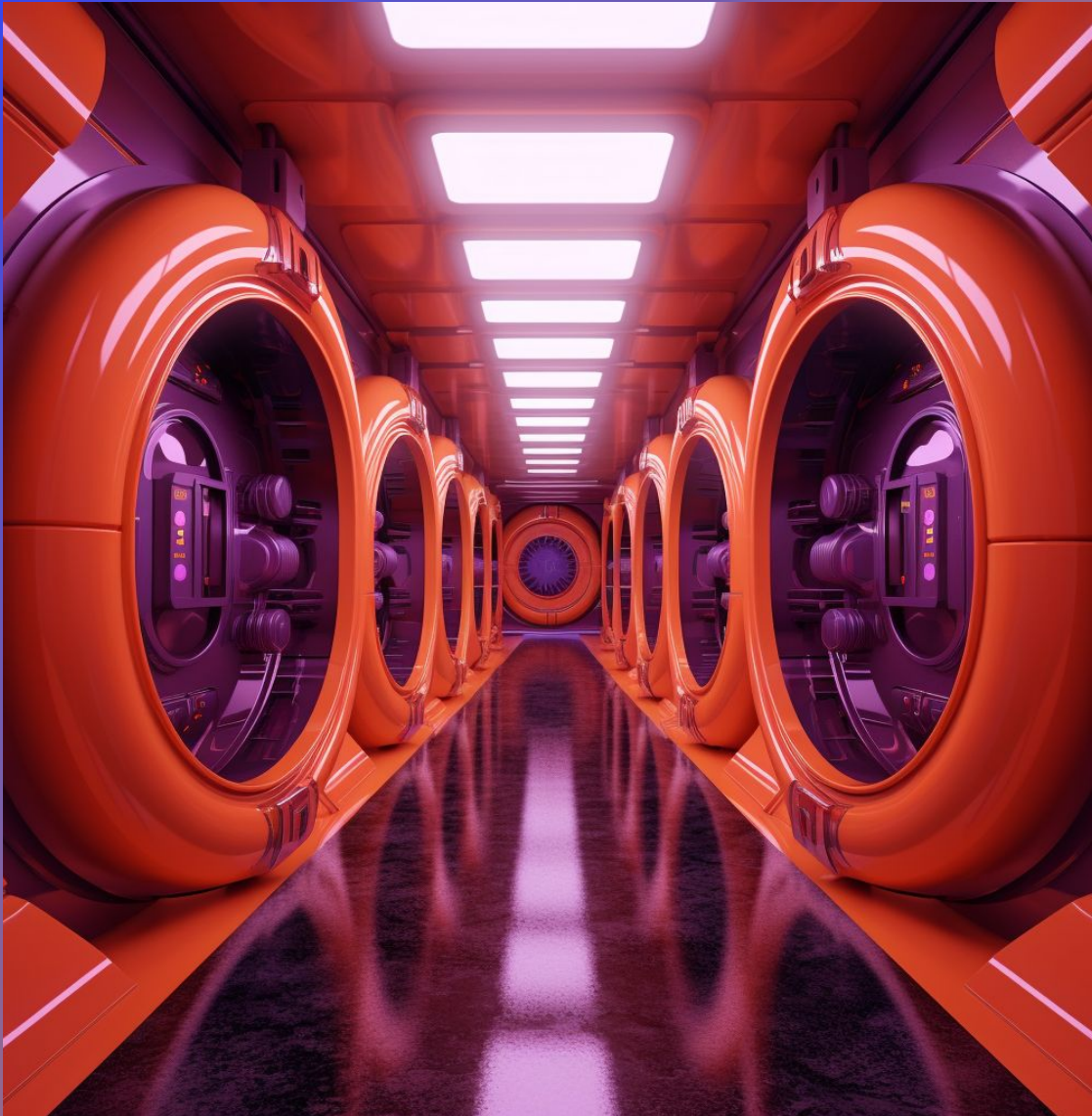
Avant d'autoriser tout retrait ou transfert de fonds, le contrat de coffre-fort vérifie que l'utilisateur est le propriétaire légitime des actifs. Cela peut être réalisé en comparant l'adresse de l'utilisateur avec celle associée aux fonds déposés.

Mécanismes de sécurité :

Le contrat de coffre-fort peut mettre en place des mécanismes de sécurité supplémentaires, tels que des mots de passe, des autorisations multi-signatures ou des délais de retrait. Ces mécanismes permettent de renforcer la sécurité en exigeant des étapes supplémentaires pour accéder aux actifs.

Retrait des fonds ou des actifs :

Les utilisateurs peuvent retirer leurs fonds ou leurs actifs en appelant une fonction spécifique de retrait du contrat de coffre-fort. Cette fonction effectue les vérifications nécessaires et transfère les actifs à l'adresse spécifiée.



PATTERN DU VOTE (VOTING)



Le pattern du vote (Voting) est utilisé pour mettre en place des mécanismes de vote décentralisés dans les contrats intelligents. Il permet aux participants de prendre des décisions collectives, telles que l'approbation de propositions, l'élection de représentants ou la gouvernance des systèmes décentralisés.

Création du contrat de vote : Ce contrat définit les règles et les mécanismes pour l'organisation et la tenue des votes.



Pattern du vote

Propagation des propositions :

Les participants peuvent soumettre des propositions à voter. Ces propositions peuvent inclure des changements de règles, des décisions importantes ou d'autres sujets soumis à la décision collective.

Inscription des participants :

Les participants doivent être inscrits pour pouvoir voter. Le contrat de vote peut exiger une inscription préalable pour s'assurer que seuls les participants autorisés peuvent participer aux votes.

Déroulement du vote :

Une fois les propositions soumises, le contrat de vote définit une période pendant laquelle les participants peuvent exprimer leur vote. Cela peut être une période prédéfinie ou jusqu'à ce qu'un certain nombre de votes soient atteints.



Pattern du vote

Types de vote :

Le contrat de vote peut prendre en charge différents types de vote, tels que les votes majoritaires, les votes pondérés ou les votes à approbation. Chaque type de vote a ses propres règles et mécanismes pour déterminer le résultat.

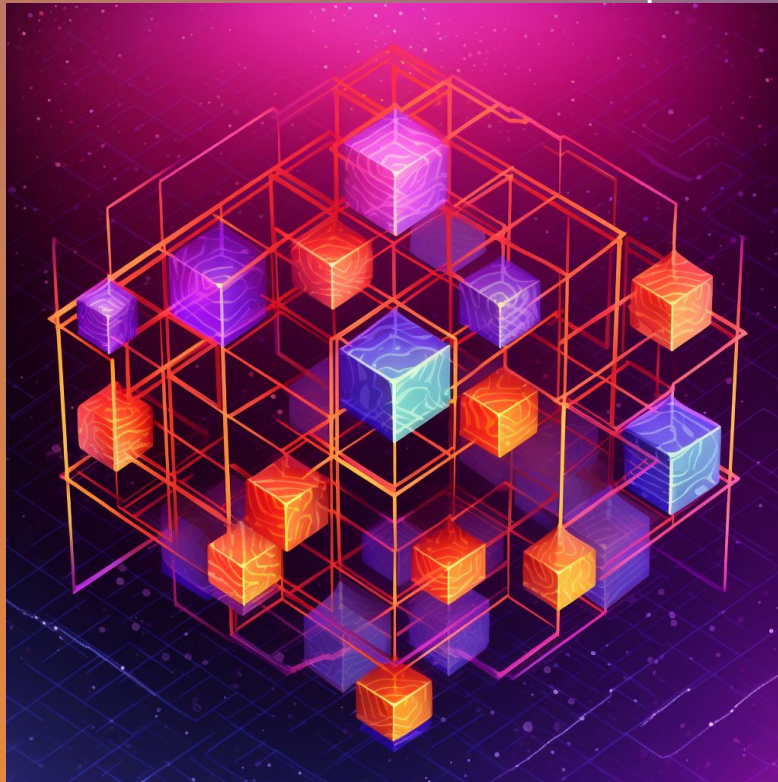
Comptage des votes et résultat :

Une fois la période de vote terminée, le contrat de vote compte les votes et détermine le résultat. Le résultat peut être basé sur la majorité des votes exprimés ou selon d'autres règles spécifiées dans le contrat.

Exécution de la décision :

Une fois le résultat du vote déterminé, le contrat de vote peut prendre des mesures en fonction de la décision collective. Cela peut inclure l'application de nouvelles règles, l'implémentation de propositions acceptées ou tout autre effet prévu par le contrat.

PATTERN DE MISE À JOUR (UPGRADE)



Le pattern de mise à jour (Upgradeability) est utilisé dans les contrats intelligents pour permettre des mises à jour et des améliorations sans perturber les fonctionnalités existantes ni nécessiter une migration complète vers un nouveau contrat.

Création du contrat de base : Ce contrat contient les fonctionnalités, les logiques initiales du système et les liens entre smart-contracts.



Pattern de mise à jour

Création du contrat de base :

Tout d'abord, un contrat de base est créé. Ce contrat contient les fonctionnalités et les logiques initiales du système.

Isolation de la logique :

La logique principale du contrat est isolée dans un ou plusieurs contrats distincts. Ces contrats, souvent appelés "contrats de logique" ou "implémentations", contiennent les fonctionnalités et les algorithmes spécifiques du système.

Utilisation d'un contrat de proxy :

Un contrat de proxy est créé pour agir comme une couche d'interface entre les utilisateurs et les contrats de logique. Le contrat de proxy reçoit les appels des utilisateurs et les transmet aux contrats de logique correspondants.





Pattern de mise à jour

Mise à jour des contrats de logique :

Lorsqu'une mise à jour ou une amélioration est requise, un nouveau contrat de logique est développé avec les modifications nécessaires. Ce nouveau contrat de logique est déployé sur la blockchain.

Changement de référence :

Une fois le nouveau contrat de logique déployé, le contrat de proxy est mis à jour pour pointer vers le nouveau contrat de logique au lieu de l'ancien. Ainsi, tous les appels ultérieurs seront redirigés vers le nouveau contrat de logique.

Continuité des fonctionnalités :

Grâce à l'utilisation du contrat de proxy, les fonctionnalités existantes sont maintenues et les utilisateurs peuvent continuer à interagir avec le système sans perturbation, tout en bénéficiant des améliorations et des nouvelles fonctionnalités du nouveau contrat de logique.

+

○

●

MERCI

Helder SALVADOR
hsalvador@esgi.fr



Exercices

